

Serveur HDS Mylbe (Épione / ISIS)

Synthèse des échanges du 2026-07-10 et 2026-07-11 : résolution d'un incident cache, remise en état des mises à jour système, audit sécurité, et planification d'une intervention de mise à jour majeure.

Contexte

Serveur de production **certifié HDS** (Hébergeur de Données de Santé), hôte `WEB-PHP-PROJECT`, Debian 11 (bullseye), hébergeant **deux applications santé mutualisées sur la même machine** :

- **Épione** — ce dépôt, Symfony 7.2, migration en cours depuis une version Symfony 4 (hébergée sur un autre serveur, avec aussi une copie locale dans `../epione`).
- **ISIS** — Symfony 6, deux tenants sur une seule appli, différenciés par environnement : **IXAIR Médical** et **ISIS Atlantique**.

Utilisateurs : plusieurs milliers de connexions/jour — admins, profils internes (techniciens, assistantes...), médecins, et patients (profil le plus exposé).

Accès : root en SSH pour les opérations OS, utilisateur `mylbe` pour les opérations applicatives (git, déploiement...).

Équipe projet : Laurent (développeur, pas administrateur système de formation, seul à intervenir sur ce serveur au quotidien), Michaël et José (partenaires) — José a installé le serveur à l'origine ; spécialiste Microsoft, peu familier de Linux mais compétent. Pas d'administrateur Linux dédié "sous la main".

Implication clé : toute intervention **au niveau OS/serveur** impacte les deux applications (Épione ET ISIS), pas seulement celle sur laquelle on travaille habituellement — le rayon d'action d'un correctif système est plus large qu'un simple souci applicatif Épione.

Incident résolu — dossier fantôme `!!!XXXX` pendant

`cache:clear`

Symptôme

En production, `APP_DEBUG=0 php bin/console cache:clear --env=epione` laissait occasionnellement un répertoire au nom bizarre (ex: `!!4XF`, `!!sEb`) dans `var/cacheef70d58b/`, sibling du dossier `epione`, contenant des restes de cache.

Hypothèses écartées

- **NFS/CIFS "silly rename"** : écarté après vérification — `findmnt -T var/cacheef70d58b/epione` confirme un filesystem **ext4 local** (`/dev/md1`), pas un montage réseau.
- **Scanner de sécurité / antivirus** (Imunify360, ClamAV...) qui quarantinerait les fichiers PHP fraîchement compilés : écarté après capture live.

Root cause confirmée

C'est **Symfony lui-même**. `Filesystem::remove()` (`vendor/symfony/filesystem/Filesystem.php`) renomme un dossier en `!! + 2 octets aléatoires (base64, inversé)` **avant** de supprimer son contenu récursivement — mécanisme de sécurité volontaire pour rendre le renommage atomique :

```
$tmpName = \dirname(realpath($file)).'/.!'.strrev(strtr(base64_encode(random_bytes(2)), '/=', '-!'))
// ... rename($file, $tmpName) puis suppression récursive du contenu de $tmpName ...
// ... puis rmdir($tmpName) si tout s'est bien passé
```

Confirmé en direct avec `inotifywait` (surveillance du **dossier parent** `var/cacheef70d58b`, pas du sous-dossier `epione`) pendant un vrai `cache:clear` :

```
epion_ CREATE,ISDIR          ← warmupDir créé
epione MOVED_FROM,ISDIR / epion~ MOVED_TO,ISDIR    ← ancien cache renommé
epion_ MOVED_FROM,ISDIR / epione MOVED_TO,ISDIR    ← nouveau cache promu
epion~ MOVED_FROM,ISDIR / !!sEb MOVED_TO,ISDIR     ← rename de sécurité avant suppression
```

Pourquoi ça reste parfois bloqué : la suppression récursive du contenu échoue occasionnellement (probablement une requête Apache `mod_php` concurrente — ce serveur tourne en `mod_php`, pas `php-fpm` — qui écrit dans l'arborescence de cache pile au moment du `clear`). L'exception `IOException` levée est **avalée silencieusement** par le `try/catch` de `CacheClearCommand` (visible uniquement en mode `-vvv`), donc `cache:clear` affiche "succès" mais laisse l'orphelin sur disque. Confirmé intermittent (pas systématique) et confirmé **non lié aux droits** : tous les dossiers de cache/log sont en `777`, `mylbe:mylbe`.

Correctif appliqué

Dans `~/epionesf7/clear-cache.sh`, juste après `cache:clear` :

```
sudo find var/cacheef70d58b -maxdepth 1 -name '.*!*' -exec rm -rf {} +
```

Pas besoin de filtre d'âge (`-mmin`) ici : à ce stade du script, `epione` pointe déjà vers le nouveau cache, donc tout `.*!xxx` restant est déjà détaché du chemin utilisé par l'appli — suppression immédiate sans risque. (Pour une version cron indépendante du script, garder une marge de sécurité type `-mmin +60` .)

Points annexes relevés (à traiter dans l'audit sécurité)

- `clear-cache.sh` fait un `sudo chmod 777 -R` sur `var/cacheef70d58b/epione` et `var/logef70d58b/epione` — dossiers world-writable sur un serveur HDS, à revoir.
- `clear-cache.sh` lance un `composer install` nu (sans `--no-dev --optimize-autoloader`) en prod avant `cache:clear` .

Blocage `apt update` résolu — clé GPG sury.org expirée

Diagnostic

`apt update` échouait uniquement sur le dépôt tiers `packages.sury.org/php` (fournit les PHP 8.1/8.2/8.4 sur Debian 11) :

```
Err: https://packages.sury.org/php bullseye InRelease
  Les signatures suivantes ne sont pas valables : EXPKEYSIG B188E2B695BD4743 ...
```

Tous les dépôts officiels (Debian bullseye, `security.debian.org` , `repo.mysql.com` , `mirrors.online.net`) fonctionnaient normalement — **pas** un problème de distribution EOL, juste une clé de signature Sury expirée (2026-02-04), un événement classique et sans gravité sur ce dépôt.

Correctif appliqué

```
curl -fsSL https://packages.sury.org/php/apt.gpg | apt-key add -
```

Fingerprint vérifié identique à l'ancien avant import (`1505 8500 A023 5D97 F5D1 0063 B188 E2B6 95BD 4743`) pour écarter tout risque de MITM sur la récupération de la clé. Clé valide jusqu'au 2028-02-04 après correctif.

Note technique : la clé est stockée dans le trousseau global déprécié (`apt-key` , pas de fichier dédié dans `/etc/apt/trusted.gpg.d/` , pas de `signed-by` dans `/etc/apt/sources.list.d/php.list`).

Migration future possible vers un keyring dédié avec `signed-by` , non urgente.

Audit sécurité / mises à jour (2026-07-11)

● Échéance critique : fin du support Debian 11 le 31 août 2026

- Bullseye (Debian 11) est sorti en août 2021, a quitté le support standard de l'équipe sécurité Debian en août 2024, tourne depuis en **LTS bénévole** (couverture réduite à un sous-ensemble de paquets) jusqu'au **31 août 2026**.
- Après cette date, `security.debian.org` cesse de patcher `oldoldstable` sauf souscription à l'**Extended LTS (ELTS) payant de Freexian** (couverture jusqu'en 2031).
- Debian 12 (bookworm) est lui-même déjà sorti de son support régulier le 10 juin 2026 → la cible naturelle d'une montée de version n'est pas Debian 12 mais **Debian 13 (trixie)**.

Sources : tuxcare.com, freexian.com, endoflife.ai

Paquets en attente

~278 paquets à mettre à jour, répartis sur les dépôts officiels (`mirrors.online.net` , `security.debian.org` , `repo.mysql.com` , `packages.sury.org`). Accumulation normale d'environ 2 ans sans `apt upgrade` .

unattended-upgrades : absent

Aucune mise à jour de sécurité automatique n'est appliquée sur ce serveur — package non installé, aucun fichier de config, aucun service. Explique le retard accumulé. Question ouverte : mettre en place une config **sécurité uniquement, sans redémarrage automatique, avec notification** (à décider, pas fait à ce stade).

Noyau

- Actif : `5.10.0-18-amd64` (paquet `5.10.140-1`)
- Disponible : `5.10.259-1` (≈ 2 ans de correctifs noyau LTS 5.10)
- Pas de `/var/run/reboot-required` actuellement (normal, la mise à jour n'est pas encore installée)
- Appliquer ce correctif nécessite un **redémarrage complet**, donc une coupure simultanée d'Épione et ISIS (pas de redondance visible sur ce serveur).

debsecan — audit CVE structuré

Installé et exécuté (`apt install debsecan && debsecan --suite bullseye`). Points clés de lecture du rapport :

- **Les CVE PHP taguées (obsolete) sont un faux positif à ignorer** : PHP vient de `sury.org` (dépôt tiers), que le *Debian Security Tracker* ne suit pas — `debsecan` ne peut pas évaluer ces paquets et les marque "obsolete" faute de correspondance. Le signal fiable pour PHP reste `apt list --upgradable`.
- **L'écrasante majorité du volume brut concerne `linux-libc-dev`** (plusieurs milliers de lignes) — reflet du même écart noyau déjà identifié (5.10.140 → 5.10.259), pas une info nouvelle.
- **Éléments réellement actionnables** (CVE non corrigées, hors noyau et PHP, avec un lien direct vers ce que font les applis) :

Paquet	Pertinence
curl / libcurl	Appels HTTP sortants PHP (HttpClient Symfony)
ImageMagick (libmagickcore/wand)	Traitement d'images uploadées — historique de RCE ("ImageTragick")
libheif1 (HEIF/HEIC)	Format photo iPhone — pertinent si upload mobile
poppler-utils / libpoppler102 (PDF)	Si génération/parsing de PDF (comptes-rendus...)
libexpat1 (XML)	Parsing XML PHP/Symfony
openssh-server/client/sftp	Accès admin distant — 8 CVE récentes sans correctif Debian dispo pour l'instant

- **Conclusion pratique** : la plupart des CVE listées comme "non corrigées" par `debsecan` seront en fait déjà résolues par le simple `apt upgrade` complet déjà identifié comme nécessaire (le correctif existe dans les dépôts, juste pas encore installé). Priorité : la fenêtre de maintenance plutôt qu'un patch paquet par paquet.

Plan de mise à jour du serveur — à venir

Risques identifiés

Risque	Détail
Services qui redémarrent mal	Config incompatible après upgrade (MariaDB, Apache, PHP), prompts <code>dpkg</code> interactifs sur les fichiers de conf modifiés
Perte d'accès SSH	<code>openssh-server</code> fait partie des paquets à mettre à jour — risque le plus critique vu l'absence d'admin "sous la main"
Coupure simultanée Épione + ISIS	Le reboot noyau coupe tout, pas de redondance
Dépôts tiers multiples	sury.org + mysql.com + Debian → risque de conflit de dépendances plus élevé qu'un <code>apt upgrade</code> Debian pur
Pas de retour arrière automatique	Aucun snapshot connu à ce stade — la récupération dépend entièrement de ce qui aura été préparé avant

Décisions prises

- **Fenêtre** : samedi matin (faible trafic, laisse jusqu'au dimanche soir pour déboguer si besoin).
- **Approche en 3 séances distinctes**, pour diluer le risque, avec une semaine d'observation entre chaque :
- **Séance 1** — paquets "sûrs" (libs sans redémarrage de service critique).
- **Séance 2** — services applicatifs (Apache, PHP, MariaDB).
- **Séance 3** — noyau + reboot (en dernier, une fois la confiance établie sur les 2 premières séances ; José/Michaël prévenus et joignables ce jour-là).
- Protection de la session pendant l'upgrade via `tmux` / `screen` (une coupure réseau ne doit pas interrompre l'opération en plein milieu).
- Vérifier la reconnexion SSH avant de considérer une étape terminée.

Clarification — accès nécessaire pour le reboot

Un reboot normal ne nécessite pas d'accès admin/hébergeur : root en SSH suffit (`reboot` ou `shutdown -r now`), le serveur redémarre proprement et SSH remonte seul.

L'accès hors-bande (console/VNC/panneau hébergeur) n'est nécessaire qu'en cas d'échec — si le serveur ne remonte pas après reboot (panne noyau, service réseau qui ne démarre pas), il faut pouvoir agir depuis l'extérieur puisque SSH ne répond plus. C'est probablement José qui détient cet accès (il a configuré l'hébergement à l'origine).

Points en attente (à confirmer avec José et Michaël)

- [] Snapshot de la VM possible côté hébergeur avant intervention ? (pas confirmé à ce stade — filet de sécurité prioritaire)
- [] José a-t-il bien accès à la console/panneau d'administration de l'hébergeur, et sera-t-il joignable pendant la fenêtre de maintenance (filet de sécurité en cas d'échec du reboot) ?
- [] Pas d'environnement de staging disponible pour tester avant la prod — à défaut, prévoir une sauvegarde manuelle avant chaque séance : dump des bases (`mysqldump`), sauvegarde de `/etc` , liste des versions de paquets installés (`dpkg -l > paquets-avant.txt`).

Commandes de référence utilisées pendant l'investigation

```
# Vérifier le type de filesystem d'un chemin
findmnt -T var/cacheef70d58b/epione

# Surveiller en direct les créations/renommages dans un dossier (parent, non récursif)
inotifywait -m --format '%T %w%f %e' --timefmt '%F %T' \
  -e create,moved_to,moved_from,delete,delete_self \
  var/cacheef70d58b

# Nettoyage des dossiers orphelins .!!XXXX laissés par Filesystem::remove()
sudo find var/cacheef70d58b -maxdepth 1 -name '!!*' -exec rm -rf {} +

# Rafraîchir la clé GPG sury.org (avec vérification de fingerprint avant de valider)
curl -fsSL https://packages.sury.org/php/apt.gpg | apt-key add -
apt-key list 2>/dev/null | grep -i -B2 sury

# Lister les paquets sécurité en attente uniquement
apt-get --just-print upgrade 2>/dev/null | grep "^Inst" | grep -i securi

# Audit CVE structuré (bien plus fiable que l'analyse manuelle des versions)
apt install debsecan
debsecan --suite bullseye

# État du noyau actif vs installé, et redémarrage en attente
uname -r
dpkg -l | grep linux-image
ls -la /var/run/reboot-required 2>/dev/null
```